

MacSoft Agent 当前系统架构与交接报告

Client、MacSoft Server、Hermes AI Service 与 Windows Runtime 实现说明

报告版本： 1.0

产品版本： MacSoft Agent 0.1.0 (stable)

源码快照日期： 2026-07-20

参考基线： 《MacSoft Server V1 Implementation Report》（阶段 0）

目录

1. 当前系统结论
2. 从阶段 0 到当前实现
3. 部署结构与数据所有权
4. Windows Runtime 与启动生命周期
5. Client、Server 与 Hermes 的完整流程
6. MacSoft Server 已实现功能与代码落点
7. Desktop 与配置流程
8. API 快速索引
9. 配置文件快速索引
10. 安全与隔离模型
11. 已知边界与技术债务
12. 新开发人员快速上手
13. 测试与交付基线
14. 关键源码地图
15. 最终交接判断

文档目的

本报告用于把 MacSoft Agent 当前已经实现的系统交给新的开发、测试或运维人员。它回答四个最重要的问题：

- 1. Client、MacSoft Server、Hermes 与 Windows Runtime 分别负责什么；
- 2. 一次配对、聊天和 AutoCount 调用实际经过哪些组件；
- 3. Server 已经实现了哪些功能，以及这些功能落在哪些代码中；
- 4. 新接手人员应从哪些配置、端口、数据目录和源码入口开始排查。

本报告描述的是 **2026-07-20 所见源码快照的实际行为**，不是阶段 0 报告中的未来计划。阶段 0 文档仍保留为设计背景；凡是两者不一致，以当前源码和当前打包配置为准。

1. 当前系统结论

MacSoft Agent 当前不是一个单进程程序，而是一套由 Windows Host 统一管理的本地服务组合：



系统的核心边界如下：

组件	主要责任	不负责的事项
Mac Soft Client	配对、保存设备身份、显示会话、发送消息、消费 SSE	不直接访问 Hermes；不决定 Server 实际使用的模型；不拥有 Server 数据
MacSoft Server	设备认证、会话隔离、消息持久化、聊天编排、SSE、Private Skill 边界	不直接实现模型/provider 登录；不直接包含 AutoCount 业务命令
Hermes AI Service	模型/provider 调用、Agent/tool loop、插件加载、AutoCount 工具执行	不直接向局域网 Client 暴露；不管理 Client 设备和会话归属
Config Backend	为 Server Desktop 提供 Hermes 设置、模型和凭证配置接口	不是 Client API；不是聊天推理端口
MacSoftAgentHost	初始化 ProgramData、启动/停止/监控三个子服务、提供本机控制接口	不处理业务会话和聊天内容

组件	主要责任	不负责的事项
Server Desktop	管理服务、网络、模型/provider、AutoCount 设置与配对码	不是外部 Thin Client；不应把 Host Control Token 暴露给 renderer

2. 从阶段 0 到当前实现

阶段 0 报告主要是一份合同优先的开发计划。当前代码已经把其中多数核心边界落地，并在部分区域超出了初始范围。

阶段 0 主题	当前状态	说明
Server 与 Thin Client API 合同	已实现	配对、认证、会话、消息、SSE 已具备
SQLite 持久化	已实现	users、devices、pairing_codes、client_skills、sessions、messages
设备级会话隔离	已实现	sessions 使用 owner_device_id, 查询同时限制 user 与 device
Hermes 统一模型出口	已实现	Server 通过本机 /v1/chat/completions 调用 Hermes
Private Skill 边界	已实现	设备隔离、内容校验、仅作为不可信偏好注入
AutoCount docs_search 初期范围	已扩展	当前插件包含查询、schema、验证与执行工具；不再只是文档搜索
durable audit_logs	未见当前实现	当前数据库初始化未创建持久审计表
HTTPS / 公网部署	未实现	当前产品以局域网 HTTP 模式运行

这意味着接手人员不能只阅读阶段 0 文档来判断现状，必须以本报告所列源码入口为主。

3. 部署结构与数据所有权

3.1 Program Files：不可变程序文件

典型安装根目录：

```
C:\Program Files\MacSoft Agent
```

主要包括：

- Server Desktop（Electron 桌面管理程序）；
- Hermes AI Service 源码/运行包；
- MacSoft Server Python 包；
- 内置 Python；
- macsoft_runtime Windows Host；
- 初始化模板与产品元数据。

升级安装时，这一层可以被新版本替换。运行时不应把客户数据写回 Program Files。

3.2 ProgramData：可写运行数据

典型数据根目录：

```
C:\ProgramData\MacSoft Agent
```

主要包括：

- Server YAML 与 Hermes runtime YAML；
- SQLite 数据库；
- Hermes provider/model/auth 运行状态；
- AutoCount 插件配置与受保护资源；
- Windows Host 控制令牌和状态；
- 日志、备份和升级时保留的数据。

`initialize_product_data()` 采用“缺失才创建”的原则处理可变配置和数据库，并用原子写入避免半写文件。受保护资源按版本和哈希更新；遇到用户修改冲突时应保留而不是静默覆盖。

3.3 源码与打包产物

当前主要源码根目录：

```
C:\MacSoft-Agent
```

外部 Client 目录：

```
C:\MacSoft-win-unpacked
```

本次检查中，外部 Client 目录表现为 Electron 已打包输出，包含 `Mac Soft.exe`、`resources\app.asar` 和压缩后的前端 bundle。它可以用于验证 API 合同和运行行为，但不等同于适合维护的 TypeScript 源码仓库。正式交接时应另行保存 Client 的原始仓库、构建脚本和提交号。

另外，C:\MacSoft-Agent 根目录在本次检查时没有可用的 .git 元数据，无法从该目录直接确认当前提交、分支或未提交差异。这是交接与追溯风险，建议后续把每次发行包绑定到不可变 commit、build manifest 和 SHA-256。

4. Windows Runtime 与启动生命周期

4.1 系统入口

安装后由 Windows Service MacSoftAgentHost 管理后台进程。Service 入口位于：

```
product_runtime\macsoft_runtime\service.py
```

启动时大致执行：

```
Windows SCM
-> MacSoftAgentHostService
-> prepare_host()
-> initialize_product_data()
-> acquire host lock
-> start Host Control
-> start_all()
```

4.2 子服务启动顺序

product_runtime\macsoft_runtime\host.py 定义三个 ServiceSpec：

顺序	服务名	默认地址	启动目的
1	config_backend	127.0.0.1:8643	为 Desktop 设置页提供 Hermes 配置后端
2	ai_service	127.0.0.1:8642	提供 Hermes Agent、模型和工具 API
3	server	0.0.0.0:8787	对局域网 Mac Soft Client 提供业务 API

依赖关系也由 Host 强制保证：

- 启动 ai_service 时，若 config_backend 未运行，会先启动它；
- 启动 server 时，若 ai_service 未运行，会先启动 AI Service；
- 停止时采用逆序：Server -> AI Service -> Config Backend。

因此，UI 中“MacSoft Server Running”只表示 8787 服务状态；模型设置一直 Loading 或 ECONNREFUSED 127.0.0.1:8643 则通常是 Config Backend/Host 问题，而不是 8787 API 本身。

4.3 Host Control

Host Control 仅监听：

```
127.0.0.1:8766
```

控制令牌写入 ProgramData 的 Host 配置 JSON。Desktop 主进程读取它，renderer 不应直接获得令牌。

主要本机接口：

方法与路径	用途
GET /v1/status	获取三个子服务状态
GET /v1/pairing-code	通过 Host 安全取得 Server 配对码
POST /v1/services/{name}/start	启动指定服务
POST /v1/services/{name}/stop	停止指定服务
POST /v1/services/{name}/restart	重启指定服务
POST /v1/autostart	设置 Windows 自动启动行为

Host 把同一个本机控制令牌分别传给：

- Config Backend：HERMES_DASHBOARD_SESSION_TOKEN；
- MacSoft Server：MACSOFT_HOST_CONTROL_TOKEN。

配对码的安全调用链是：

```
Server Desktop
-> Host Control :8766
-> MacSoft Server loopback /api/internal/pairing-code
-> pairing_codes table
```

局域网客户端不能直接调用开发配对码接口；公开的 /api/dev/pairing-code 当前故意返回 404。

5. Client、Server 与 Hermes 的完整流程

5.1 首次安装与 Desktop 打开

1. 安装器放置 Program Files 程序文件并注册 MacSoftAgentHost。
2. Host 或 Desktop 调用 `python -m macsoft_runtime --initialize-only`。
3. initializer 创建缺失的 ProgramData 目录、配置、数据库和共享本机 API key。
4. Windows Service 拉起 8643、8642、8787。
5. Server Desktop 通过 Host Control 获取状态，并连接 8643 加载模型/provider 设置。

Desktop 设置页面的保存操作只更新配置；启动、停止、重启由 Host Control 单独执行，避免“保存设置”隐式改变服务状态。

5.2 Client 首次配对

Client 请求：

```
POST /api/client/pair
Content-Type: application/json

{
  "pairing_code": "123456",
  "device_id": "<client-generated-stable-id>",
  "client_name": "<display name>",
  "client_version": "<version>"
}
```

Server 行为：

1. 检查配对码存在、未使用、未过期；
2. 以 BEGIN IMMEDIATE 原子事务占用配对码，避免两个 Client 同时领取；
3. 接受 Client 提交的 device_id，不生成新的 device ID；
4. 生成 device token 并写入 devices；
5. 返回 snake_case 与 camelCase 兼容字段。

成功响应的关键字段：

```
{
  "ok": true,
  "device_id": "<accepted client device id>",
  "deviceId": "<accepted client device id>",
  "device_token": "<generated token>",
  "deviceToken": "<generated token>",
  "paired_user": "<display name>",
  "pairedUser": "<display name>",
  "role": "<role>",
  "paired_at": "<timestamp>",
  "pairedAt": "<timestamp>"
}
```

已打包 Client bundle 的实际调用证据显示，它提交自己的 device.device_id，并同时兼容读取 device_token/deviceToken 与 device_id/deviceId。

5.3 已配对 Client 认证

后续请求使用：

```
Authorization: Bearer <device token>
X-Device-Id: <device id>
```

Server 验证：

- token 对应的设备必须存在且处于 active；
- 对应用户必须 active；
- 若提供 X-Device-Id，必须与 token 所属设备一致。

GET /api/client/me 返回 Server 所认可的用户、角色、设备和模型信息。Client 的 preferred_model_id 只是兼容输入，当前 Server 不让 Client 决定真实 provider/model。

5.4 会话创建与隔离

Client 使用：

```
GET    /api/sessions
POST   /api/sessions
GET    /api/sessions/{session_id}/messages
DELETE /api/sessions/{session_id}
```

Server 的所有会话查询都同时使用：

```
user_id + owner_device_id + deleted_at IS NULL
```

所以同一 Server 上的不同 Client 设备不会看到彼此会话。删除采用 soft delete，只写 deleted_at，不会把 provider 凭证作为会话数据删除。

历史旧数据迁移规则：

- 如果旧数据库只有一个 active device，可以安全回填该设备；
- 如果存在多个 active device，无法确定归属的旧会话保持未分配，避免错误泄露给任一设备。

5.5 聊天与 SSE

打包 Client 发送：

```
POST /api/chat/stream
Authorization: Bearer <device token>
X-Device-Id: <device id>
Accept: text/event-stream
X-MacSoft-Client-Capabilities: activity-v1
```

请求体包含：

```
{
  "session_id": "<session id>",
  "message": "<user message>",
  "preferred_model_id": null,
  "enabled_private_skills": [],
  "uploaded_file_ids": [],
  "client_info": {
    "device_id": "<device id>",
    "app_version": "<version>"
  }
}
```

Server 内部流程：

认证设备

- > 校验 session 属于当前 user + device
- > 在 ActiveChatRunRegistry 中占用 session
- > 再次确认 session 未被删除
- > 保存 user message
- > 构建有上限的历史上下文
- > 解析当前设备选中的 Private Skills
- > 注入受保护的系统边界
- > 调用 Hermes :8642 /v1/chat/completions
- > 把 Hermes tool progress 映射成 activity-v1
- > 格式化和安全处理最终回复
- > 先保存 assistant message
- > 再发送最终 token_delta 与 message_done
- > 释放 ActiveChatRunRegistry

关键设计点：

- 同一个 session 同时只能有一个 active run；
- 不同 session 可以并发运行；
- 正在回复的 session 不能被删除，避免“生成中删除”竞态；
- assistant 回复必须先成功写入 SQLite，才向 Client 宣告最终完成；
- generator 正常结束、错误或 Client 断开时都会尝试释放占用；
- activity 文本会去除路径、token、secret 和 traceback 等敏感细节；
- Client 端是否能正确同时显示多个 session 的流，是 Client 状态隔离问题；Server 负责持久化和 session 归属。

主要 SSE 事件：

事件	含义
message_start	本次 assistant 消息开始
activity	工具或处理阶段状态，只有支持 activity-v1 时发送
token_delta	可显示的回复增量
error	当前生成或持久化失败
message_done	本次消息结束，ok 表示是否成功

5.6 Hermes 与模型调用

Server 不直接连接 OpenAI 或其他 provider。它调用：

```
http://127.0.0.1:8642/v1/chat/completions
```

并使用 initializer 生成的共享本机 API key。Hermes API Server 提供 OpenAI-compatible 请求格式，然后由 Hermes 内部完成：

- provider/model 选择；
- 认证信息读取；
- Agent loop；
- toolset 加载；
- AutoCount 工具调用；
- streaming content 与受控 tool progress 输出。

Server 的 hermes_client.py 负责：

- 规范化 messages；
- 加入 Bearer key；
- 区分 auth、unavailable、timeout、protocol 等错误来源；
- 解析普通 content delta；
- 只接受受控 hermes.tool.progress 事件作为工具进度；
- 拒绝空回复和协议异常。

5.7 AutoCount 调用

AutoCount 不是直接写在 server\macsoft 中的业务模块，而是 Hermes runtime 插件：

```
runtime\plugins\macsoft-autocount
```

插件向 Hermes 注册以下工具：

工具	作用
autocount_get_connector_status	检查 Cloud/Connector/Company 状态
autocount_search_commands	搜索官方命令目录
autocount_get_command_schema	获取指定命令的精确 schema
autocount_validate_command	在提交前验证 payload
autocount_execute_command	重新解析、验证并执行命令

典型执行链：

用户请求

- > Server 受保护系统指令
- > Hermes Agent
- > 搜索/解析精确官方 command
- > 获取 schema
- > 校验 payload
- > 检查 connector
- > POST command
- > 轮询结果
- > Hermes 返回结果
- > Server result_formatter 转成可读 Markdown
- > 保存并 SSE 给 Client

插件没有把每个 AutoCount 命令硬编码为独立 Python 函数，而是依赖 Cloud 的动态 catalog/schema。安全行为包括：

- 命令必须精确解析，近似名称只提供建议；
- 执行前重新解析与重新验证；
- GET 可对短暂错误做一次有界重试；
- POST 不自动重试，避免重复写入；
- 相同无效 payload 有 fingerprint 抑制；
- 异常内容在返回 Client 前被清理。

需要注意两个配置层：

1. Server YAML 中的 `autocount.enabled` 当前也用于 Server 自身状态/配置；
2. Hermes runtime YAML 中的 `plugin/toolset` 才决定 AI Service 是否加载 `macsoft-autocount`。

交接时不能只看 Server /health 中的 `autocount_enabled` 就断言 Hermes 工具是否加载成功。

6. MacSoft Server 已实现功能与代码落点

6.1 应用入口、配置与健康检查

功能	实现位置
FastAPI app、路由装配、CORS、startup DB 初始化	server\macsoft\server.py
YAML dataclass 配置	server\macsoft\config.py
/health	server\macsoft\gateway\routes_health.py
统一错误处理	server\macsoft\errors.py

Server 监听 YAML 中的 host/port，当前模板为 0.0.0.0:8787。这允许同一局域网访问，但 Windows 防火墙规则仍要与当前网络 Profile 匹配。

CORS 当前显式允许的是本地开发 renderer 来源 (5173/5174)。已打包 Electron Client 使用 file://，通常不会携带浏览器网页的 Origin。若未来把 Client 改成普通网页或公网 Web App，必须重新设计允许来源与 HTTPS，而不是简单开放 *。

6.2 数据库与迁移

实现位置：

```
server\macsoft\db.py
```

当前表：

表	用途
users	用户、角色、active 状态
devices	Client 设备、device token、最后活动
pairing_codes	配对码、过期时间和使用状态
client_skills	设备级 Private Skill
sessions	会话、owner device、soft delete
messages	user/assistant 消息与状态

init_db() 每次启动确保 schema 存在，并执行 owner device / client skill 的兼容迁移。当前快照未见阶段 0 规划的 durable audit_logs 表。

6.3 配对与设备认证

实现位置：

```
server\macsoft\gateway\routes_client.py
server\macsoft\auth.py
```

已实现：

- 内部配对码只允许 loopback + Host Control Bearer；

- 公开开发配对码接口关闭;
- 6 位配对码与过期时间;
- atomic pairing claim;
- 接受并保存 Client device ID;
- snake_case / camelCase 配对响应兼容;
- Bearer device token 认证;
- X-Device-Id 绑定检查;
- active device 与 active user 检查。

6.4 Session 与消息

实现位置:

```
server\macsoft\gateway\routes_sessions.py
server\macsoft\sessions\session_store.py
server\macsoft\sessions\message_store.py
```

已实现:

- 设备级 session ownership;
- list/create/messages/delete;
- soft delete 与幂等删除;
- 正在生成时返回 busy;
- message preview 和 session touch;
- AI 历史窗口限制。

上下文窗口当前大致限制为:

- 最多 41 条消息;
- 估算最多 12,000 tokens;
- 最多扫描 123 个候选历史消息;
- 只保留完整 user/assistant 对, 再加当前 user message。

这样做避免无限把整个数据库 transcript 送入模型, 并防止历史截断到半个对话回合。

6.5 Chat 编排与 SSE

实现位置:

```
server\macsoft\gateway\routes_chat.py
server\macsoft\chat\active_runs.py
server\macsoft\chat\hermes_client.py
server\macsoft\chat\activity.py
server\macsoft\chat\capability_policy.py
server\macsoft\chat\result_formatter.py
```

已实现:

- 空白消息拒绝;
- 文本长度/UTF-8 字节上限;
- attachment/uploaded file 明确拒绝;

- session active-run reservation;
- delete/run 竞态二次确认;
- Hermes streaming bridge;
- legacy Client 与 activity-v1 Client 兼容;
- 工具活动去敏;
- 持久化优先于 final done;
- 错误来源分类与用户可读 Markdown;
- 对识别出的 AutoCount JSON/business rows 转为表格/摘要;
- 对实时新闻、天气、市场、汇率、交通和体育等能力声明设置保护边界。

当前没有获批准的实时互联网工具时，系统不应假装已经取得实时数据。Private Skill、Public 描述文本或用户消息都不能自行授予工具权限。

6.6 Private Device Skills

实现位置：

```
server\macsoft\gateway\routes_skills.py
server\macsoft\skills\client_skills.py
```

已实现：

- list、validate、create、get、update、delete;
- user + owner_device_id 双重隔离;
- slug/content 大小与数量限制;
- 最多选择 5 个;
- 选中内容总量限制;
- 拒绝控制字符、路径穿越、可执行脚本/代码围栏;
- 拒绝索取 token/secret;
- 拒绝覆盖 Protected/Public 规则;
- 拒绝借 Skill 声称 AutoCount 工具授权。

被选中的 Private Skills 只在该次请求中被编码为“不可信偏好”，不会注册新工具，也不会改变服务端权限。

7. Desktop 与配置流程

Server Desktop 基于 Hermes Desktop/Electron，但产品模式下由 Windows Host 管理后台，不由 Desktop 自己随意拉起一套平行服务。

主要代码：

```
hermes\apps\desktop\electron\macsoft-product.ts
hermes\apps\desktop\electron\macsoft-product-initializer.ts
hermes\apps\desktop\electron\macsoft-host-client.ts
hermes\apps\desktop\electron\server-autocount-config.ts
hermes\apps\desktop\electron\main.ts
```

职责：

- 解析 Program Files/ProgramData 产品路径;
- 启动时确认产品数据初始化完成;
- 读取 Host Control 连接信息;
- 查询三个服务状态;
- 取得配对码;
- 启动、停止、重启服务;
- 加载/保存 Server、Hermes、AutoCount 配置;
- 枚举网卡并生成 Client URL;
- 测试 Server、AI Service 与 AutoCount 连接;
- 使用备份和原子替换保存设置。

典型故障归因：

现象	首选检查
Model 页面持续 Loading，提示 8643 refused	Host Service、Config Backend、Host token 文件
Server 显示 Stopped，8787 无响应	Host 状态、Server log、端口占用、YAML
本机 health 正常，另一电脑连接失败	Windows Network Profile、防火墙入站规则、实际 WLAN IP
配对码拿不到但 /health 正常	Host Control -> internal pairing route，而不是普通 health
Client 认证失败	device token、device ID、device active 状态
模型凭证异常	Hermes/Config Backend 数据；不要先删除 session DB

8. API 快速索引

8.1 对外 Client API（8787）

方法	路径	认证	用途
GET	/health	否	Server 基础健康检查
POST	/api/client/pair	配对码	首次设备配对
GET	/api/client/me	Device Bearer	当前设备与用户
GET	/api/sessions	Device Bearer	当前设备会话列表
POST	/api/sessions	Device Bearer	创建会话
GET	/api/sessions/{id}/messages	Device Bearer	当前设备会话消息
DELETE	/api/sessions/{id}	Device Bearer	soft delete 会话
POST	/api/chat/stream	Device Bearer	SSE 聊天
GET/POST	/api/client/skills	Device Bearer	Private Skills
POST	/api/client/skills/validate	Device Bearer	校验 Skill
GET/PATCH/DELETE	/api/client/skills/{slug}	Device Bearer	单项 Skill 操作

8.2 本机内部接口

端口	路径/协议	调用方	用途
8766	/v1/*	Desktop main process	Windows Host Control
8643	Hermes config backend	Desktop	模型、provider、凭证和设置
8642	/v1/chat/completions	MacSoft Server	Hermes AI/Agent
8787	/api/internal/pairing-code	Host Control loopback	安全生成/获取配对码

9. 配置文件快速索引

配置	模板源码	运行时位置/用途
产品元数据	product.json	version、build id、schema、runtime base
Hermes runtime	packaging\templates\runtime\config.yaml	model/provider、8642、plugin/toolset
MacSoft Server	packaging\templates\server\macsoft-server.yaml	8787、SQLite、Hermes URL/key
AutoCount plugin	packaging\templates\protected\runtime\plugins\macsoft-autocount	Cloud URL、API key、connector/company、工具实现
Host Control state	由 runtime 创建	8766 端口和控制 token
初始化状态	由 initializer 创建	本机共享 API key、资源版本和哈希

共享本机 key 使用 `${MACSOFT_LOCAL_API_KEY}` 模板替换，不应硬编码进源码或发行 manifest。

10. 安全与隔离模型

当前系统的主要安全边界：

1. **网络边界**：只有 8787 对局域网监听；8642、8643、8766 均为 loopback。
2. **设备边界**：Device Bearer token + 可选 X-Device-Id。
3. **数据边界**：session 和 Private Skill 都包含 owner_device_id。
4. **配对边界**：pairing code 一次性、过期、原子占用；生成接口只走本机 Host。
5. **工具边界**：Client 和 Private Skill 不能注册工具或改变 Server/Hermes 权限。
6. **输出边界**：activity、错误和 AutoCount 结果在对 Client 输出前进行清理/格式化。
7. **文件边界**：Program Files 放程序；ProgramData 放可变配置、凭证和数据库。

当前局限：

- 8787 是 HTTP，不是 HTTPS，不适合直接暴露到公网；
- device token 与 pairing code 当前以 SQLite 数据字段保存，未见 hash-at-rest；
- active run registry 是进程内状态，当前部署默认单 Server 进程；若未来多 worker/多节点，必须改成共享锁或数据库协调；
- Host Control 的安全依赖 loopback 与本机 token 文件；
- 普通卸载/升级必须正确保留 ProgramData，显式 purge 才应删除数据。

11. 已知边界与技术债务

以下不是本报告要立即修改的 bug，而是交接时必须知道的边界：

1. **公网部署未就绪**。需要 HTTPS、反向代理、明确 Origin、证书、速率限制和更完整的身份体系。
2. **Client 原始源码不在当前 Client 目录**。当前目录适合运行和检查 bundle，不利于长期维护。
3. **根源码目录缺少 Git 追溯信息**。无法直接证明发行包对应哪个 commit。
4. **凭证/设备 token 的 at-rest 保护仍可加强**。当前重点是功能边界与本机隔离，不应误称为完整企业密钥管理。
5. **多进程扩展需要重新设计 run lock**。当前 in-memory registry 适用于单进程 Server。
6. **附件未实现**。uploaded_file_ids 非空时会被拒绝，避免表现为“上传成功但模型没收到”。
7. **模型由 Hermes/Server 所有**。Client 发来的 preferred model 不具权威性。
8. **持久审计表未落地**。需要审计合规时应设计独立 append-only audit，不要把普通日志当成等价品。
9. **AutoCount 是动态高权限能力**。执行功能已存在，生产上线前仍需按客户权限、确认流程和真实 Cloud 环境做业务验收。
10. **两层 AutoCount 状态可能引起误判**。Server YAML 与 Hermes plugin/toolset 要分别检查。

12. 新开发人员快速上手

12.1 第一小时阅读顺序

建议按以下顺序阅读，不要从 Electron UI 大包开始：

1. product.json
2. product_runtime\macsoft_runtime\paths.py
3. product_runtime\macsoft_runtime\initializer.py
4. product_runtime\macsoft_runtime\host.py
5. product_runtime\macsoft_runtime\control.py
6. server\macsoft\server.py
7. server\macsoft\db.py
8. server\macsoft\gateway\routes_client.py
9. server\macsoft\gateway\routes_sessions.py
10. server\macsoft\gateway\routes_chat.py
11. server\macsoft\chat\hermes_client.py
12. hermes\gateway\platforms\api_server.py
13. packaging\templates\protected\runtime\plugins\macsoft-autocount
14. Desktop 的 macsoft-host-client.ts 与 server-autocount-config.ts

12.2 最小运行诊断

建议按链路逐层检查：

1. Windows Service 是否存在/运行
2. 127.0.0.1:8766 Host Control 是否可用
3. 127.0.0.1:8643 Config Backend 是否健康
4. 127.0.0.1:8642 Hermes AI Service 是否健康
5. 127.0.0.1:8787/health 是否健康
6. 服务器实际 WLAN/Ethernet IP:8787/health 是否健康
7. 另一台电脑到该 IP:8787/health 是否健康
8. 配对
9. /api/client/me
10. session create/list/messages
11. chat SSE
12. AutoCount connector/schema/execute

不要从第 12 步的业务错误反推第 1 步；也不要只因 /health 成功就断言 Hermes provider 已经可用。

12.3 改动原则

- Server 合同变更先检查已打包 Client bundle 与正式 Client 源码；
- device_id、session owner、Host token 与 local API key 的所有权不能混淆；
- 更改启动顺序时同时检查 dependency、health probe、stop order；
- 更改 ProgramData 时同时检查 fresh install、preserved reinstall、normal uninstall、purge uninstall；
- 更改聊天流时同时检查保存顺序、SSE 事件、Client 断开、delete busy 与多 session；
- 更改 AutoCount 时优先保持 catalog/schema 驱动，不把客户命令硬编码进 Server；
- 任何“修复”都应有 focused test，并验证没有把安全边界转移给 Client。

13. 测试与交付基线

当前源码中可见的主要自动化测试：

范围	代表测试
配对与身份	test_pairing_security.py
设备归属迁移	test_device_ownership_migration.py
Session 删除/并发	test_session_soft_delete.py
Chat 边界	test_chat_boundaries.py
Activity/SSE	test_activity_protocol.py
Capability policy	test_capability_policy.py
Client Skills	test_client_skills.py
CORS	test_cors.py
AutoCount validator	test_autocount_validator.py
Product paths/init/host	test_product_paths.py、test_initializer.py、test_host.py
Packaging/staging	test_packaging_contract.py、test_staging.py
Desktop Host integration	macsoft-host-client.test.ts、macsoft-product-initializer.test.ts 等

交接时建议把以下信息写入每次发行记录：

- Git commit;
- product.json build id;
- staging manifest;
- installer size 与 SHA-256;
- Python/TypeScript/packaging test 结果;
- fresh install、preserved-data upgrade、normal uninstall、purge uninstall;
- 实际 Client pairing、multi-session、SSE、AutoCount acceptance。

14. 关键源码地图

子系统	关键路径
产品元数据	C:\MacSoft-Agent\product.json
Windows Host	C:\MacSoft-Agent\product_runtime\macsoft_runtime\host.py
Host Control	C:\MacSoft-Agent\product_runtime\macsoft_runtime\control.py
Windows Service	C:\MacSoft-Agent\product_runtime\macsoft_runtime\service.py
ProgramData 初始化	C:\MacSoft-Agent\product_runtime\macsoft_runtime\initializer.py
路径模型	C:\MacSoft-Agent\product_runtime\macsoft_runtime\paths.py
Server 入口	C:\MacSoft-Agent\server\macsoft\server.py
数据库	C:\MacSoft-Agent\server\macsoft\db.py
配对/设备	C:\MacSoft-Agent\server\macsoft\gateway\routes_client.py
Sessions	C:\MacSoft-Agent\server\macsoft\gateway\routes_sessions.py
Chat/SSE	C:\MacSoft-Agent\server\macsoft\gateway\routes_chat.py
Hermes bridge	C:\MacSoft-Agent\server\macsoft\chat\hermes_client.py
Private Skills	C:\MacSoft-Agent\server\macsoft\gateway\routes_skills.py
Hermes API Server	C:\MacSoft-Agent\hermes\gateway\platforms\api_server.py
Desktop Host client	C:\MacSoft-Agent\hermes\apps\desktop\electron\macsoft-host-client.ts
Desktop config	C:\MacSoft-Agent\hermes\apps\desktop\electron\server-autocount-config.ts
Runtime template	C:\MacSoft-Agent\packaging\templates\runtime\config.yaml
Server template	C:\MacSoft-Agent\packaging\templates\server\macsoft-server.yaml
AutoCount plugin	C:\MacSoft-Agent\packaging\templates\protected\runtime\plugins\macsoft-autocount
已打包 Client	C:\MacSoft-win-unpacked\Mac Soft.exe
Client bundle	C:\MacSoft-win-unpacked\resources\app.asar

15. 最终交接判断

当前系统已经形成一条完整的核心业务链：

安装与初始化

- > Windows Host 管理本机服务
- > Client 安全配对
- > 设备级 Session 隔离
- > Server 持久化消息并编排请求
- > Hermes 统一处理模型和工具
- > AutoCount 通过动态 schema 受控执行
- > Server 清理、保存并以 SSE 返回

对下一位维护者而言，最重要的理解不是“每个文件做什么”，而是三个所有权：

1. Client **拥有显示状态和稳定 device ID，但不拥有 Server 数据与模型决策；**
2. MacSoft Server **拥有设备身份、session、消息、边界与对 Client 的响应合同；**
3. Hermes **拥有 provider/model、Agent loop 与工具执行，Windows Host 拥有本机进程生命周期。**

只要修改时保持这三个所有权不被打乱，系统就可以在现有核心功能之上继续演进，而不需要通过跨层补丁“拆东墙补西墙”。

报告依据

- 阶段 0 参考文档：C:\MacSoft-Agent\docs\MacSoft_Server_V1_Implementation_Report.docx
- 当前 Server/Runtime/Hermes 源码：C:\MacSoft-Agent
- 当前已打包 Client 接口证据：C:\MacSoft-win-unpacked
- 审阅日期：2026-07-20